Listing 2.8 `prompt_engineering.py (main())`

```
def main():
    directory = "prompts"
    text_files = list_text_files_in_directory(directory)    #1

    if not text_files:
        print("No text files found in the directory.")
        return

    def print_available():                                  #2
        print("Available prompt tactics:")
        for i, filename in enumerate(text_files, start=1):
            print(f"{i}. {filename}")

    while True:
        try:
            print_available()                   #2
            choice = int(input("Enter … 0 to exit): "))           #3
            if choice == 0:
                break
            elif 1 <= choice <= len(text_files):
                selected_file = text_files[choice - 1]
                file_path = os.path.join(directory,
      selected_file)
                prompts =
↪ load_and_parse_json_file(file_path)                          #4
                print(f"Running prompts for {selected_file}")
                for i, prompt in enumerate(prompts):
                    print(f"PROMPT {i+1} --------------------")
                    print(prompt)
                    print(f"REPLY ---------------------------")
                    print(prompt_llm(prompt))                        #5
            else:
                print("Invalid choice. Please enter a valid number.")
        except ValueError:
            print("Invalid input. Please enter a number.")
```

#1 Collects all the files for the given folder
#2 Prints the list of files as choices
#3 Inputs the user's choice
#4 Loads the prompt and parses it into messages
#5 Submits the prompt to an OpenAI LLM

A commented-out section from the listing demonstrates how to connect to a local LLM. This will allow you to explore the same prompt engineering tactics applied to open source LLMs running locally. By default, this example uses the OpenAI model we configured previously in section 2.1.1. If you didn't complete that earlier, please go back and do it before running this one.

Figure 2.10 shows the output of running the prompt engineering tactics tester, the `prompt_engineering.py` file in VS Code. When you run the tester, you can enter a value for the tactic you want to test and watch it run.